



Universidad Experimental De Guayana

Vicerrectorado Académico

Coordinación General De Pregrado

Proyecto De Carrera: Ingeniería En Informática

Asignatura: Ingeniería Del Software II

Sección 1

Gestión de Riesgos y Deuda Técnica

Profesor:

Márquez Félix

Grupo: Ingenio y Código

Aguilera Rhixeidys V-30.851.503

Bravo Yonkeiner V-30.994.057

Carreño Brayan V-32.015.527

Garcia Nicole V-30.809.865

Ciudad Guayana, septiembre de 2026

Introducción

La evolución de la ingeniería de software moderna ha transformado la gestión de riesgos, pasando de ser un proceso burocrático, documental y estático a convertirse en una disciplina proactiva, empírica y adaptativa. El presente informe detalla la aplicación práctica de estos conceptos contemporáneos en ecosistemas tecnológicos altamente distribuidos, a través de la resolución de tres ejercicios fundamentales.

En primer lugar, se analiza un caso de estudio centrado en una aplicación Fintech sustentada en Inteligencia Artificial y una arquitectura Serverless, donde se identifican y mitigan amenazas críticas modernas como la deuda técnica, vulnerabilidades en la cadena de suministro, el vendor lock-in y los riesgos ético-legales derivados de los sesgos algorítmicos. En segundo lugar, se aborda la dicotomía existente entre la rigurosidad de las auditorías corporativas y la agilidad operativa, proponiendo el diseño de un Registro de Riesgos Híbrido (One-Page Risk Register) que se integra orgánicamente en las ceremonias del marco de trabajo Scrum. Finalmente, el documento establece una comparativa analítica entre las sesiones tradicionales de identificación de riesgos y las prácticas ágiles de Risk Storming, demostrando cómo el enfoque multifuncional y colaborativo resulta indispensable para auditar la complejidad de las arquitecturas actuales y transformar las amenazas cualitativas en acciones técnicas tangibles.

Ejercicio 1: Fintech con IA y Arquitectura Serverless

- **Contexto del Proyecto:**

Desarrollo de una aplicación móvil Fintech de préstamos, soportada por una API de Inteligencia Artificial (IA) de un tercero para evaluar riesgo crediticio y alojada íntegramente en un modelo Serverless con un único proveedor de nube.

1. Identificación de Riesgos (Risk Storming Simulado)

En entornos ágiles, la identificación de riesgos se realiza mediante el "Risk Storming", una sesión colaborativa visual donde todo el equipo democratiza la detección de amenazas analizando la arquitectura. A continuación, se identifican 4 riesgos críticos según las categorías modernas solicitadas:

- **Riesgo (a) Técnico / Deuda Técnica:**

Decisión de acoplar directamente la lógica de negocio a la API de terceros (solución rápida y subóptima) en lugar de construir una capa de abstracción adecuada, generando una deuda técnica temprana. Esto acelerará el time-to-market inicialmente, pero reducirá la velocidad de desarrollo futura y aumentará la fragilidad del sistema.

- **Riesgo (b) Ciberseguridad / Cadena de Suministro:**

Integración de bibliotecas de código abierto (opensource) con vulnerabilidades heredadas no detectadas (CVEs) en las funciones Serverless, lo cual expone a la plataforma a ataques en la cadena de suministro.

- **Riesgo (c) Vendor Lock-in (Dependencia Crítica):**

Dependencia excesiva y exclusiva de los servicios propietarios del proveedor Serverless (Ejemplo: funciones, bases de datos no relacionales, IAM). Esto dificultaría técnica y financieramente una migración futura hacia otra nube.

- **Riesgo (d) Ético o Legal (IA):**

Introducción de sesgos algorítmicos opacos por parte de la API de IA de terceros, resultando en rechazos discriminatorios de créditos. Esto derivaría en incumplimientos regulatorios, demandas legales y un severo daño en la reputación de la empresa.

2. Análisis Cualitativo del Riesgo Crítico

Para este ejercicio, evaluaremos el **Riesgo (d) - Sesgo algorítmico ético/legal de la IA**, ya que la IA introduce comportamientos no estocásticos y opacos que comprometen el negocio principal de la Fintech (aprobación de créditos).

El análisis cualitativo utiliza una matriz ordinal de Probabilidad-Impacto para categorizar el evento rápidamente y enfocar los recursos organizacionales:

- **Probabilidad (4 - Alta):**

Al depender de un modelo de caja negra (API de tercero) sin control directo sobre sus datos de entrenamiento, la probabilidad de comportamiento no estocástico o sesgado es muy alta según las advertencias de marcos de gobernanza como NIST AI RMF 1.0.

- **Impacto (5 - Crítico):**

Un fallo discriminatorio en los préstamos generaría daños reputacionales irreversibles, acciones legales e incumplimientos regulatorios que amenazan la continuidad operativa de la Fintech.

- **Cálculo del Risk Score:**

Utilizando la fórmula clásica de Exposición = Probabilidad × Impacto.

- **Risk Score:** $4 \times 5 = 20$ (Prioridad Crítica).

3. Estrategias de Mitigación y Respuestas al Riesgo

Dado el umbral de riesgo crítico (Score 20), debemos planificar una respuesta inmediata.

La estrategia recomendada será un enfoque dual de **Mitigación** y **Transferencia**:

3.1 Transferencia (Contractual/Financiera):

Exigir un Acuerdo de Nivel de Servicio (SLA) estricto con el proveedor de la API de IA que incluya cláusulas de indemnización y cumplimiento de normativas de equidad algorítmica. Esto actúa como un instrumento de gestión que desplaza la responsabilidad operativa al proveedor.

3.2 Mitigación (Técnica/Interna):

Dado que la mitigación se aplica cuando el equipo tiene control sobre la vulnerabilidad o falla arquitectónica, el equipo no puede reescribir el modelo de IA del tercero, pero sí puede controlar cómo consume el servicio. La mitigación será aplicar marcos de gobernanza de IA para garantizar explicabilidad.

4. Traducción a Tarea Tangible en el Backlog Ágil

Para evitar que los riesgos sean un simple ejercicio burocrático relegado a un plan estático de Word o Excel, la metodología ágil exige que se conviertan en trabajo técnico visible en el Product Backlog.

La mitigación técnica propuesta se traduce en los siguientes artefactos vivos para el equipo Scrum:

- **Spike de Investigación (Sprint actual):**

- Título: Investigar e integrar marco de auditoría de explicabilidad para la API de créditos.
 - Descripción: El equipo dedicará un 'Timebox' de 3 días para investigar cómo implementar una validación intermedia que audite los resultados devueltos por el proveedor de IA y aplique un marco de gobernanza, con el objetivo de prevenir sesgos antes de otorgar el crédito. Esto democratiza la detección de amenazas con el equipo.
- **Historia de Habilitación / Enabler (Siguiendo Sprints):**
 - Título: Implementar capa 'Shadow Testing' para evaluación crediticia.
 - Descripción: Como Arquitecto de Software, necesito que las decisiones de la API de IA sean almacenadas y evaluadas estadísticamente en paralelo (Shadow mode) sin afectar al usuario final durante un mes, para que podamos demostrar la ausencia de sesgos demográficos antes de liberar la versión final a producción.

Esta aproximación asegura que la gestión de riesgos no solo identifique problemas, sino que modifique proactivamente el código y la arquitectura construyendo resiliencia organizacional de forma empírica.

Ejercicio 2: Diseño de un Registro de Riesgos Híbrido y Gobernanza Ágil

1. Introducción y Conflicto Operacional: Auditoría vs. Agilidad

En la ingeniería de software moderna, la gestión de riesgos se enfrenta recurrentemente a una dicotomía cultural y metodológica. Por un lado, las instancias directivas, comités de auditoría y estándares de aseguramiento de la calidad (tales como ISO 31000:2018, PMBOK 7ma Edición e ISO/IEC/IEEE 12207) exigen evidencia formal, rigurosidad documental,

identificación de propietarios y trazabilidad determinista del riesgo para salvaguardar la continuidad operativa y mitigar la responsabilidad corporativa. Por otro lado, los equipos de desarrollo fundamentados en marcos ágiles como Scrum priorizan el software funcionando, la iteración rápida y la minimización de actividades que no generen valor directo para el usuario final (Cohn, 2005).

La consecuencia habitual de esta tensión es la generación de 'cementorios de riesgos': voluminosas matrices en hojas de cálculo (Excel) creadas durante las etapas iniciales de licitación o kickoff que jamás vuelven a ser consultadas por el equipo de desarrollo. Este enfoque estático incumple el principio fundamental de la ingeniería de software propuesto por Boehm (1991), el cual concibe la gestión de riesgos como un proceso dinámico de aprendizaje y control continuo. El presente trabajo resuelve esta brecha mediante un enfoque híbrido, donde los requerimientos de gobernanza y auditoría formal se operacionalizan dentro de los artefactos y ceremonias vivas de Scrum, garantizando cumplimiento normativo sin imponer sobrecarga burocrática al flujo de entrega.

2. Diseño del Registro de Riesgos Híbrido (ONE-PAGE RISK REGISTER)

Para conciliar las demandas de auditoría externa con la agilidad operativa, se ha diseñado un Registro de Riesgos Híbrido de una sola página. Este artefacto sintetiza la estructura canónica de la norma ISO 31000 con los elementos de flujo de trabajo de Scrum. A continuación, se describen los campos normativos y conectores ágiles que estructuran la herramienta:

- **ID Único del Riesgo:** Identificador alfanumérico secuencial e inmutable (ej. RSK-01) que asegura la trazabilidad en actas de auditoría y commits de control de versiones.

- **Descripción Estructurada (Causa - Evento - Consecuencia):** Redacción formal bajo la sintaxis estándar: 'Debido a [Causa/Vulnerabilidad], podría ocurrir [Evento de Riesgo], resultando en [Impacto operativo o financiero]'.
- **Evaluación Cualitativa (Probabilidad e Impacto):** Escala cuantitativa-cualitativa normalizada de 1 a 5, donde el Nivel de Exposición o Risk Score se calcula mediante la relación matemática formal: $\text{Risk Score} = \text{Probabilidad (P)} \times \text{Impacto (I)}$, con rango de 1 a 25.
- **Dueño del Riesgo (Risk Owner):** Rol específico del Scrum Team (Product Owner, Scrum Master, Tech Lead o Desarrollador Senior) legalmente responsable de monitorear el indicador y velar por la ejecución de la respuesta.
- **Estrategia de Respuesta (PMBOK):** Clasificación formal de la respuesta ante amenazas: Mitigar (reducir P o I), Evitar (eliminar la causa raíz), Transferir (compartir el riesgo con terceros o seguros) o Aceptar (asumir el impacto pasiva o activamente).
- **Conector Ágil al Backlog (Puente Híbrido):** Enlace directo y bidireccional al Product Backlog mediante el identificador del ticket de trabajo (User Story técnica, Technical Spike o criterio en la Definition of Done - DoD).
- **Estado de Ciclo de Vida:** Estado vivo del riesgo: Identificado, En Mitigación Activa, Bajo Monitoreo, Controlado/Cerrado o Materializado.

En la Tabla 1 se presenta la matriz operativa del Registro de Riesgos Híbrido. Obsérvese cómo los riesgos no son abstracciones archivadas, sino elementos directamente vinculados a ítems del backlog y épicas técnicas del proyecto corporativo.

Tabla 1. Matriz de Registro de Riesgos Híbrido Auditable (One-Page Risk Register)

ID	Descripción del Riesgo (Causa → Evento → Consecuencia)	P (1-5)	I (1-5)	Score (P×I)	Estrategia y Dueño	Conector Ágil (Artefacto Backlog)	Estado
RSK-01	Dependencia de API externa privativa → Proveedor modifica tarifas o TOS intempestivamente → Sobrecosto severo o corte del servicio.	4	4	16 (Alto)	Mitigar Owner: Tech Lead (Brayan C.)	SPIKE-04: Evaluar modelos OSS locales. ENABLER-12: Gateway de IA con Fallback.	En mitigación (Sprint 4)
RSK-02	Falta de aislamiento en arquitectura multi-tenant → Fuga de datos de pacientes veterinarios → Incumplimiento legal y multas regulatorias.	2	5	10 (Medio)	Evitar Owner: Security Champion	ENABLER-08: Row Level Security (RLS) en Postgres + Criterio en DoD.	Controlado (Sprint 2)
RSK-03	Agotamiento de cuotas y timeout en cálculo de dosis críticas → Latencia >10s en quirófano → Error médico y daño reputacional.	3	5	15 (Alto)	Mitigar Owner: Backend Dev	ENABLER-15: Caché distribuido Redis offline-first para fórmulas.	Planificado (Sprint 5)
RSK-04	Rotación de desarrolladores clave con conocimiento centralizado → Pérdida de know-how en algoritmos → Retraso >3 semanas en entregas.	3	3	9 (Medio)	Mitigar Owner: Scrum Master	TASK-22: Pair Programming rotativo y documentación en ADRs.	Bajo monitoreo

Nota: Los niveles de severidad se rigen por la matriz de calor estándar: Verde (1-6, Riesgo Bajo), Amarillo (8-12, Riesgo Medio) y Rojo (15-25, Riesgo Alto/Crítico). Cualquier riesgo clasificado en zona roja exige automáticamente la creación de un ítem técnico en el Backlog del Sprint inmediato.

3. Mecanismo de Gobernanza Ágil: Mantener el Registro Vivo Sin Burocracia

Uno de los postulados centrales del Manifiesto Ágil es la valoración de los 'individuos e interacciones sobre procesos y herramientas' y la 'respuesta ante el cambio sobre seguir un plan'. Para que la gestión de riesgos no degenere en una carga administrativa, se establece una política de cero reuniones adicionales (Zero Extra Meetings Policy). La gobernanza del riesgo se fusiona orgánicamente dentro de los eventos e interacciones que Scrum ya estipula:

- **Sprint Planning (Planificación del Sprint) - Reserva de Capacidad y Mitigación**

Proactiva:

Durante la primera parte de la sesión de planificación, el Product Owner y el equipo de desarrollo revisan el estado de los riesgos en zona roja o amarilla. Se adopta la práctica recomendada por Kruchten et al. (2012) y Cohn (2005) de reservar entre un 15% y un

20% de la capacidad de desarrollo (Sprint Capacity) específicamente para la reducción de deuda técnica, investigaciones arquitectónicas (Spikes) y tareas de mitigación de riesgos (Enablers). De este modo, mitigar un riesgo no es un favor no planificado, sino un compromiso explícito dentro del Sprint Goal.

- **Daily Scrum (Sincronización Diaria) - Sensor de Alerta Temprana:**

La reunión diaria de 15 minutos opera como un radar operacional. Se orienta al equipo a incorporar una perspectiva preventiva en la discusión de impedimentos. Si un desarrollador detecta una anomalía (por ejemplo, cambios imprevistos en la respuesta de un servicio externo, fallos en la integración continua o degradación del rendimiento), el riesgo se verbaliza al instante como un impedimento. El Scrum Master registra el evento en el tablero vivo sin esperar al cierre de la iteración.

- **Backlog Refinement (Refinamiento Continuo) - Risk-Driven Prioritization:**

En las sesiones de refinamiento, el equipo dimensiona las Historias de Usuario evaluando no solo el esfuerzo, sino la incertidumbre técnica. Si una historia presenta una alta exposición a fallos, se descompone preventivamente mediante un Technical Spike (timeboxed de 2 a 3 días) para despejar las dudas antes de que la historia ingrese a un sprint de construcción definitivo.

- **Sprint Review (Revisión y Demostración) - Transparencia Auditable ante Stakeholders:**

La demostración del incremento no se limita a características visuales. El equipo presenta ante los interesados y auditores las 'evidencias objetivas de mitigación': pruebas de carga ejecutadas exitosamente, reportes automatizados de análisis estático (SonarQube), dashboards de control presupuestario o simulacros de contingencia. Esto satisface plenamente la necesidad de evidencia auditable requerida por la dirección.

- **Sprint Retrospective (Retrospectiva) - Inspección y Adaptación del Registro:**

Los últimos 10 minutos de la retrospectiva se reservan exclusivamente para la 'Revisión del Risk Register'. El equipo responde tres preguntas estructuradas: (1) ¿Las mitigaciones implementadas en el sprint lograron reducir el Risk Score de los riesgos asignados? (2) ¿Se materializó algún riesgo imprevisto durante el incremento? (3) ¿Qué nuevos riesgos técnicos o de negocio vislumbramos para los próximos sprints? El Risk Owner actualiza los estados en el tablero digital en tiempo real, garantizando que el registro refleje siempre el estado fáctico del software.

4. Caso Práctico: Transformación de Riesgo en Artefacto Ágil (ENABLER)

A continuación, se desarrolla el escenario planteado por la cátedra: 'La API de IA podría cambiar sus términos de servicio y costos de licenciamiento'. Se demuestra cómo este riesgo cualitativo se desglosa y materializa en código entregable mediante un proceso de ingeniería estructurado en dos fases:

4.1 Análisis y Registro Formal de la Amenaza (Nivel Gobernanza)

- Contexto Operativo:

La aplicación depende de una API externa en la nube (ej. OpenAI / Claude / Gemini) para el procesamiento inteligente de datos. Existe una vulnerabilidad crítica de 'Vendor Lock-in' (acoplamiento rígido de código al SDK del proveedor) y exposición a volatilidad tarifaria unilateral.

Evaluación Inicial en el Registro:

- **Probabilidad:** 4 (Alta, dado el dinamismo y ajustes frecuentes en políticas de APIs de IA comerciales).
- **Impacto:** 4 (Crítico, un incremento tarifario o corte de servicio paraliza la propuesta de valor y amenaza la solvencia financiera).

- **Risk Score Inicial:** $4 \times 4 = 16$ (**Zona Roja / Máxima Prioridad**).

Estrategia Seleccionada: Mitigación Arquitectónica Activa y Desacoplamiento (Transferencia/Aceptación descartadas por inviables).

4.2. Fase 1: Technical Spike de Investigación Arquitectónica (Sprint N)

Dado que el equipo desconoce la mejor estrategia para abstraer el proveedor sin introducir latencia excesiva, no se compromete directamente una historia de producción. En su lugar, el Product Owner prioriza un Spike Técnico con tiempo delimitado (Timebox):

- **TICKET:** SPIKE-IA-01 | Tipo: Technical Spike | Estimación: 3 Story Points (Timebox: 2 días)
- **Título:** Investigación y prototipo de capa agnóstica para servicios de IA y evaluación de contingencia.
- **Objetivo de Ingeniería:** Diseñar una interfaz desacoplada bajo el patrón Adapter/Facade que permita intercambiar el proveedor de IA en tiempo de ejecución y evaluar la viabilidad de un modelo de código abierto local (ej. Llama 3 / Mistral vía Ollama) como fallback ante fallos o costos excesivos.
- **Entregable Obligatorio:** Registro de Decisión Arquitectónica (ADR-04) documentado y rama técnica de pruebas con métricas de latencia y costo comparativo.

4.3. Fase 2: User Story / Enabler de Implementación en Producción (Sprint N+1)

Con los resultados empíricos obtenidos en el Spike, el Product Owner incorpora un Enabler de Arquitectura en el Product Backlog. Este ítem compete y se prioriza formalmente en el Sprint Planning junto a las historias funcionales del negocio:

- **TICKET:** ENABLER-IA-02 | Tipo: Enabler de Arquitectura y Resiliencia | Estimación: 8 Story Points

- **Título:** Implementación de Gateway Adaptador de IA y Disyuntor Presupuestario (Budget Circuit Breaker).
- **Narrativa de Usuario Formal:**
 - **Como:** Arquitecto de Software y Líder de Operaciones del Sistema,
 - **Quiero:** Implementar un patrón de diseño Adapter/Strategy desacoplado junto con un middleware de control de cuotas y gasto en tiempo real,
 - **Para:** Garantizar que el sistema pueda conmutar transparentemente entre múltiples proveedores de IA (o modelos locales de contingencia) y bloquear automáticamente peticiones anómalas si se alcanza el 80% del presupuesto diario asignado, evitando sobrecostos y cortes del servicio.

Criterios de Aceptación Técnicos (Formato BDD / Gherkin):

- **Escenario 1:** Conmutación transparente por degradación de servicio o falla tarifaria
 - Dado que el proveedor principal de IA devuelve un código de error HTTP 429 (Rate Limit), 402 (Payment Required) o latencia > 4000ms,
 - Cuando el cliente interno ejecute una solicitud de inferencia crediticia o clínica,
 - Entonces el sistema conmuta automáticamente al proveedor secundario de respaldo en menos de 500ms, registrando una advertencia en el log de auditoría sin interrumpir la experiencia de usuario.
- **Escenario 2:** Límite presupuestario estricto (Cost Circuit Breaker)
 - Dado que el consumo acumulado de tokens alcanza el 85% del presupuesto diario fijado (\$15 USD/día),
 - Cuando se reciba una nueva solicitud de procesamiento no crítico,
 - Entonces el sistema activa el disyuntor, conmuta las consultas al modelo optimizado local y envía una alerta de alta prioridad a Slack/Email al Product Owner.

- **Escenario 3:** Trazabilidad de términos de servicio y privacidad
 - Dado que las políticas de datos del proveedor de IA exigen no enviar identificadores personales (PII),
 - Cuando el payload sea procesado por el adaptador,
 - Entonces un interceptor debe anonimizar y enmascarar automáticamente nombres, cédulas y datos confidenciales antes de emitir la petición externa.
- **Definition of Done (DoD) Específico para Mitigación de Riesgos:**
 1. Cobertura de pruebas unitarias sobre la interfaz del adaptador $\geq 90\%$.
 2. Simulación de fallo (Chaos Test) ejecutada en staging validando el fallback exitoso.
 3. Métricas de consumo diario de tokens integradas en el dashboard de Grafana/CloudWatch.
 4. Actualización obligatoria del Registro de Riesgos: el Risk Score de RSK-01 se reduce formalmente de 16 (Alto) a 4 (Bajo: P=2, I=2).

Ejercicio 3: Compare la efectividad de una sesión tradicional de identificación de riesgos (liderada únicamente por el Gerente de Proyectos con una lista de verificación) frente a una sesión de Risk Storming con todo el equipo multifuncional. ¿Por qué este último enfoque es más adecuado para detectar riesgos técnicos y de ciberseguridad en la arquitectura del software?

En la práctica actual de la ingeniería de software, el desarrollo ha experimentado una transición estructural acelerada. La industria ha abandonado definitivamente la construcción de sistemas monolíticos y aislados, evolucionando hacia la orquestación de ecosistemas hiperconectados, altamente distribuidos e interdependientes. En la actualidad, la viabilidad operativa de los proyectos recae en la integración de capas de abstracción externas: infraestructura delegada en la nube, adopción masiva de librerías de código abierto y modelos

de Inteligencia Artificial. Esta extrema complejidad exige que la gestión de riesgos deje de ser un ejercicio burocrático relegado a un "Plan de Riesgos" estático que rara vez se actualiza, para convertirse en una disciplina proactiva, empírica y adaptativa. A continuación, se presenta el análisis comparativo sobre cómo las metodologías abordan esta nueva realidad técnica.

3.1 Comparativa de Efectividad: Sesión Tradicional vs. Risk Storming

Al evaluar la efectividad de las metodologías de identificación de riesgos, se encuentran deficiencias críticas en el modelo predictivo clásico, contrastándolas fuertemente con las prácticas ágiles contemporáneas.

- **El Enfoque Tradicional (Centralizado en el Gerente de Proyectos):** En una sesión tradicional, el proceso es unidireccional. El Gerente de Proyectos asume la carga analítica apoyándose en técnicas estructuradas como listas de verificación (checklists) basadas en lecciones aprendidas, análisis SWIFT o el método Delphi. Su efectividad es sumamente limitada en el contexto moderno porque promueve la creación de artefactos estáticos (matrices de Word o Excel) que no reflejan la volatilidad del desarrollo. Además, al depender de un individuo cuya visión suele orientarse a riesgos de cronograma, presupuesto y alcance, el proyecto queda completamente ciego ante amenazas técnicas profundas, fallos de integración y vectores de ataque que no están tipificados en proyectos pasados.
- **El Enfoque Ágil (Risk Storming Multifuncional):** En contraparte, el *Risk Storming* demuestra una efectividad operativa superior al democratizar la detección de amenazas. Esta práctica consiste en una sesión de trabajo colaborativa y visual donde todo el equipo multifuncional (desarrolladores, analistas de QA y el Product Owner) se reúne frente a un diagrama de arquitectura o un flujo de usuario. Su alta efectividad radica en la sinergia de perspectivas: el Product Owner evalúa el impacto en el negocio, QA

anticipa flujos extremos que podrían corromper el sistema, y el equipo de desarrollo audita la viabilidad técnica y la seguridad de la infraestructura subyacente.

3.2 Adecuación del Risk Storming para Riesgos de Ciberseguridad y Arquitectura

Frente a la interrogante del planteamiento, se concluye que el *Risk Storming* es el único enfoque capaz de proteger el software moderno. A mayor abstracción operativa, mayor es la superficie de ataque y la complejidad en la gestión del riesgo. Un gerente tradicional no posee el contexto para auditar esta superficie; se requiere el criterio técnico de los desarrolladores. Desglosamos esta afirmación en tres áreas críticas:

A. Gobernanza en la Nube y Modelo de Responsabilidad Compartida: Las arquitecturas modernas exigen despliegues Cloud, lo que nos obliga a operar bajo el Modelo de Responsabilidad Compartida. Aunque el proveedor (AWS, Azure, GCP) asegura la infraestructura base y la red física, la organización consumidora es legal y operativamente responsable de la configuración, la protección de los datos y la gestión de identidades (IAM). Una sesión tradicional liderada por un gerente jamás detectará una política IAM excesivamente permisiva. En un *Risk Storming*, al mapear visualmente el despliegue, el equipo DevOps y de backend puede identificar de inmediato malas configuraciones en la nube pública y proponer la adopción de una Arquitectura Zero Trust (NIST SP 800-207) para mitigar amenazas de exfiltración de datos.

B. Mitigación en la Cadena de Suministro de Software (Supply Chain) e IA: La velocidad de entrega actual depende del uso intensivo de bibliotecas de terceros y la adopción de Inteligencia Artificial. Esto introduce vectores de falla complejos: los atacantes ya no vulneran el código directo, sino que explotan dependencias de código abierto (CVEs) o inyectan código malicioso, como evidenciaron los incidentes de Log4j y SolarWinds. Asimismo, la IA introduce comportamientos no estocásticos y opacos, como

sesgos, envenenamiento de datos y alucinaciones (NIST AI RMF 1.0). Detectar estos riesgos exige una auditoría exhaustiva que solo aflora cuando los desarrolladores analizan el ecosistema en conjunto durante el *Risk Storming*. Es aquí donde el equipo técnico puede exigir marcos de gobernanza y la implementación de un SBOM (Software Bill of Materials) para auditar cada componente de terceros introducido al proyecto.

C. Visibilidad, Medición y Cuantificación de la Deuda Técnica: La deuda técnica, conceptualizada por Ward Cunningham, representa la decisión de elegir una solución técnica subóptima para acelerar el *time-to-market*, generando un interés acumulativo que incrementa los costos de mantenimiento y la fragilidad del sistema. En un modelo predictivo, esta deuda permanece invisible hasta que provoca interrupciones graves en el servicio. El *Risk Storming* proporciona el escenario ideal para que los desarrolladores transparenten estas decisiones, utilizando el Cuadrante de la Deuda Técnica de Martin Fowler (prudente/imprudente, deliberada/inadvertida) para catalogarla como un riesgo de negocio inminente. A partir de esta discusión visual, el equipo puede proponer el uso del método SQALE y su índice TDR (Technical Debt Ratio), junto con herramientas como SonarQube, para cuantificar financieramente el costo de refactorización.

A modo de conclusión, se enfatiza que la superioridad metodológica del *Risk Storming* culmina en su integración directa con la ejecución del proyecto. Mientras que en las metodologías predictivas los riesgos quedan asentados pasivamente en un documento, el enfoque ágil garantiza una acción correctiva. Los riesgos de ciberseguridad, vulnerabilidades de terceros o acumulación de deuda técnica detectados por el equipo se transforman inmediatamente en trabajo tangible, ingresando al Product Backlog en forma de *Spikes* (investigaciones técnicas) o Historias de Habilidad (*Enablers*). Esto asegura que la mitigación reciba una asignación de esfuerzo real (por ejemplo, el 20% del esfuerzo del sprint)

en las iteraciones venideras, protegiendo tanto la competitividad del producto como la continuidad operativa del negocio

Conclusión

A partir del análisis desarrollado, se concluye que la gestión de riesgos en la ingeniería de software actual exige abandonar las posturas predictivas centralizadas para adoptar enfoques colaborativos y dinámicos. Las arquitecturas modernas, caracterizadas por el despliegue en la nube bajo el Modelo de Responsabilidad Compartida, la integración de librerías de terceros y la adopción de modelos de Inteligencia Artificial, introducen vectores de vulnerabilidad que no pueden ser detectados eficazmente por un único rol gerencial mediante listas de verificación tradicionales.

En este contexto, prácticas como el Risk Storming han demostrado ser superiores al democratizar la detección de amenazas, aprovechando el criterio técnico integral de desarrolladores, analistas de calidad y dueños de producto. Asimismo, se evidencia que es posible conciliar los requerimientos formales de auditoría con las metodologías ágiles mediante la implementación de un Registro de Riesgos Híbrido. Este instrumento permite que las amenazas identificadas, así como la deuda técnica acumulada, no permanezcan como métricas ignoradas, sino que se conviertan de inmediato en trabajo técnico visible y priorizado dentro del Product Backlog en forma de Spikes o Historias de Habilidad (Enablers). En definitiva, al gestionar la deuda técnica y las dependencias externas como auténticos riesgos de negocio, las organizaciones logran proteger la sostenibilidad de su arquitectura, asegurar el cumplimiento regulatorio y garantizar la viabilidad a largo plazo del producto de software.

Referencias Bibliográficas

- Boehm, B. W. (1991). Software Risk Management: Principles and Practices. IEEE Software, 8(1), 32–41. <https://doi.org/10.1109/52.62930>
- Cohn, M. (2005). Agile Estimating and Planning. Prentice Hall PTR.
- Cunningham, W. (1992). The WyCash Portfolio Management System. ACM SIGPLAN OOPS Messenger, 4(2), 29–30. <https://doi.org/10.1145/157709.157715>
- Fowler, M. (2009). Technical Debt Quadrant. Martinfowler.com. <https://martinfowler.com/bliki/TechnicalDebtQuadrant.html>
- ISO. (2018). ISO 31000:2018 Risk Management — Guidelines. International Organization for Standardization.
- Kruchten, P., Nord, R. L., & Ozkaya, I. (2012). Technical Debt: From Metaphor to Theory and Practice. IEEE Software, 29(6), 18–21. <https://doi.org/10.1109/MS.2012.167>
- Project Management Institute. (2021). A Guide to the Project Management Body of Knowledge (PMBOK Guide) – 7th Edition. Project Management Institute, Inc.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.